

# GinRummy

## Informe de laboratorio - Programación Lógica, Obligatorio 2

**Grupo 30**

**Guillermo Franco**

**Joaquin Guani**

**Lukas Kasek**

**Sergio Klein**

## 1. Introducción

El objetivo del laboratorio fue implementar en Prolog una versión simplificada de Gin Rummy para dos jugadores, denominada GinRummyLog. La solución modela cartas, manos, combinaciones válidas o melds, cálculo de deadwood y tres estrategias de juego: random, greedy y pro. En cada turno, la estrategia decide si robar del mazo o del descarte, qué carta descartar y si cortar o continuar.

El enfoque general fue separar el problema en dos partes. Primero se implementaron los predicados de análisis de mano, que permiten reconocer sets y runs, generar descomposiciones de una mano y elegir la que minimiza el deadwood. Luego se implementaron las estrategias usando esos predicados como base para tomar decisiones durante la simulación.

Las principales dificultades fueron generar descomposiciones de manos sin producir ramas infinitas, evitar usar cartas en más de un meld y diseñar una estrategia pro que utilizara información adicional sin volver excesivamente costosa la ejecución. Para resolverlo se generaron directamente subconjuntos candidatos a meld, filtrando por largo y validez antes de continuar la búsqueda sobre el resto de la mano.

## 2. Representación y predicados principales

Las cartas se representaron con términos de la forma `c(Valor, Palo)`, donde Valor puede ser `a`, `2`, ..., `10`, `j`, `q` o `k`, y Palo puede ser `c`, `d`, `t` o `p`. Una mano se representó como una lista de cartas. El orden de la lista no se considera relevante para determinar melds o deadwood.

### 2.1 Reconocimiento de melds

El predicado `is_meld/1` delega en `is_set/1` e `is_run/1`. Un set se valida comprobando que la lista tenga largo 3 o 4, que todas las cartas compartan el mismo valor y que sus palos sean distintos. Un run se valida seleccionando iterativamente la carta mínima y verificando que las cartas restantes sigan el orden consecutivo del mismo palo.

```
is_meld(Cartas) :- is_set(Cartas).
is_meld(Cartas) :- is_run(Cartas).
```

### 2.2 Cálculo de deadwood

El predicado `valor_deadwood/2` suma los valores individuales de las cartas sobrantes. El As vale 1, las cartas numéricas conservan su valor y `j`, `q`, `k` valen 10. La implementación recorre la lista con un acumulador, lo que mantiene el cálculo simple y determinista cuando la lista de cartas está instanciada.

```
valor_deadwood(Cartas, Valor) :- suma_valores(Cartas, 0, Valor).
```

### 2.3 Generación y selección de melds

Para `get_melds/3` se generan directamente subconjuntos candidatos de la mano. Cada candidato se filtra primero por largo, considerando únicamente grupos de entre 3 y 10 cartas, y luego se valida con `is_meld/1`. Si el subconjunto es un meld válido, sus cartas se

eliminan de la mano mediante `restar_listas/3` y se continúa recursivamente sobre las cartas restantes. El caso base deja las cartas que no fueron utilizadas como sobrantes, permitiendo también la descomposición sin melds. Este enfoque evita construir particiones arbitrarias que luego serían descartadas y reduce la cantidad de combinaciones inútiles exploradas.

```
get_melds(Mano, [Meld|Melds], Sobrantes) :-  
    subconjunto(Mano, Meld),  
    length(Meld, L), L >= 3, L <= 10,  
    is_meld(Meld),  
    restar_listas(Mano, Meld, RestoMano),  
    get_melds(RestoMano, Melds, Sobrantes).  
  
get_melds(Mano, [], Mano).
```

El predicado `best_melds/4` recopila las descomposiciones producidas por `get_melds/3` y se queda con aquella que minimiza el valor de deadwood. Además se controla que las cartas incluidas en los melds no superen el límite de 10 cartas, condición relevante cuando se evalúan manos de 11 cartas luego del robo.

```
best_melds(Mano, MejorMelds, Sobrante, Valor) :-  
    findall((M,S), get_melds(Mano,M,S), Melds),  
    best_melds_rec(Melds, MejorMelds, Sobrante, Valor, [], [], 111).
```

## 3. Estrategias implementadas

### 3.1 Estrategia random

La estrategia random no utiliza heurística. Para robar elige aleatoriamente entre mazo y descarte. Para descartar selecciona aleatoriamente una carta de la mano de 11 cartas. Para cerrar también toma una decisión aleatoria, pero respeta la legalidad: si intenta cortar con deadwood mayor a 10, se transforma la decisión en continuar.

### 3.2 Estrategia greedy

La estrategia greedy toma decisiones locales orientadas a reducir el deadwood inmediato. Al robar, compara el deadwood óptimo de la mano actual contra el deadwood óptimo de la mano que resultaría de tomar el tope del descarte. Solo roba del descarte si esa opción mejora el valor de deadwood.

Al descartar, greedy evalúa todas las cartas posibles a tirar. Para cada opción forma la mano resultante de 10 cartas, calcula `best_melds/4` sobre esa mano y elige el descarte que deja menor deadwood inmediato. Esta decisión sigue siendo greedy porque optimiza solamente el resultado local del turno, sin planificar robos futuros ni usar cartas vistas. Para cerrar, corta siempre que el deadwood sea menor o igual a 10.

```
descartar(OldMano, _, greedy, NewMano, NewDescarte) :-  
    findall((Deadwood, Carta, ManoRestante),  
        (select(Carta, OldMano, ManoRestante),  
         best_melds(ManoRestante, _, _, Deadwood)),  
        Opciones),  
    sort(Opciones, [(_, NewDescarte, NewMano)|_]), !.
```

### 3.3 Estrategia pro

La estrategia pro intenta mejorar greedy incorporando una función de puntaje que combina deadwood actual y potencial futuro:

```
Puntaje is Deadwood * 10 - Potencial.
```

Un puntaje menor es mejor. El deadwood se pondera por 10 para priorizar manos con menor valor muerto, mientras que el potencial resta puntos cuando las cartas sobrantes tienen posibilidades de formar melds futuros. El potencial se calcula buscando cartas cercanas: cartas del mismo valor para formar sets, cartas anteriores o siguientes del mismo palo para formar runs, y cartas no vistas que todavía podrían aparecer.

Al robar, pro simula qué ocurriría si toma el descarte y luego descarta la mejor carta según su propia heurística. Compara el puntaje de la mano actual con el puntaje resultante de esa simulación y toma el descarte solo si mejora. Al descartar, evalúa todas las cartas posibles a descartar, calcula el puntaje de la mano resultante y elige la opción de menor puntaje. En caso de empate, se favorece descartar la carta de mayor valor.

Para cerrar, pro mantiene la misma regla legal de cierre: si el deadwood es mayor a 10 continúa, y si es menor o igual a 10 corta. De esta forma, la diferencia principal frente a greedy queda concentrada en las decisiones de robo y descarte, donde pro considera una evaluación más amplia de la mano y de las cartas vistas.

## 4. Validación y mediciones

Se ejecutó el archivo test.pl provisto con SWI-Prolog. La ejecución final reportó 18 tests pasados, 0 fallidos, 0 errores y 0 warnings. El log de tests indica que todos los tests se completaron en 0,154 segundos, con 0,031 segundos de CPU.

| Métrica            | Valor   |
|--------------------|---------|
| Tests pasados      | 18      |
| Tests fallidos     | 0       |
| Errores            | 0       |
| Warnings           | 0       |
| Tiempo total tests | 0,154 s |
| CPU                | 0,031 s |

Además de los tests automáticos, se usaron ejecuciones del simulador en modo debug para verificar que los predicados principales terminan en tiempo razonable durante partidas completas. Las 50 partidas de evaluación fueron parseadas correctamente y no generaron warnings ni casos marcados para revisar.

Como consultas representativas para medición directa con time/1 se utilizaron predicados centrales sobre manos de ejemplo, tales como is\_meld/1, best\_melds/4, robar/5 y descartar/5. Las consultas son reproducibles en SWI-Prolog desde la carpeta de insumos, cargando ginrummylog.pl.

```
time(is_meld([c(2,c), c(a,c), c(3,c)]))).
time(best_melds([c(a,t),c(2,t),c(3,t),c(9,p),c(j,t)], M, S, V)).
time(descartar([c(a,t),c(2,t),c(3,t),c(9,p),c(j,t),c(5,c),c(6,d),c(7,p),c(8,t),c(k,c),c(q,d)], [], pro, NM, D)).
```

## 5. Evaluación experimental

La evaluación experimental comparó greedy contra random y pro contra greedy. Para cada comparación se ejecutaron 25 partidas con semillas 1 a 25. Las partidas se ejecutaron en modo debug para poder contar las iteraciones como apariciones de la línea "=== Turno jugador" en cada log. No hubo partidas marcadas para revisar y todas las simulaciones fueron parseadas correctamente.

| Comparación      | Victorias            | Empates | Prom. turnos | Prom. DW J1 | Prom. DW J2 |
|------------------|----------------------|---------|--------------|-------------|-------------|
| greedy vs random | greedy 25 - random 0 | 0       | 17,96        | 6,08        | 56,12       |
| pro vs greedy    | pro 16 - greedy 9    | 0       | 12,96        | 14,04       | 19,36       |

Los resultados muestran que greedy domina claramente a random, ganando las 25 partidas. En la comparación pro contra greedy, pro obtuvo 16 victorias contra 9. Esto muestra que la heurística de potencial y cartas vistas aportó una mejora frente a una greedy que ya evalúa todos los descartes posibles por deadwood inmediato.

### 5.1 Detalle greedy vs random

| Seed | Iter. | Cortó     | DW J1 | DW J2 | Ganador   | Estr. ganadora |
|------|-------|-----------|-------|-------|-----------|----------------|
| 1    | 29    | jugador 1 | 8     | 72    | jugador 1 | greedy         |
| 2    | 19    | jugador 1 | 5     | 69    | jugador 1 | greedy         |
| 3    | 29    | jugador 1 | 1     | 47    | jugador 1 | greedy         |
| 4    | 17    | jugador 1 | 7     | 76    | jugador 1 | greedy         |
| 5    | 29    | jugador 1 | 10    | 51    | jugador 1 | greedy         |
| 6    | 23    | jugador 1 | 1     | 40    | jugador 1 | greedy         |
| 7    | 15    | jugador 1 | 7     | 68    | jugador 1 | greedy         |
| 8    | 13    | jugador 1 | 3     | 60    | jugador 1 | greedy         |
| 9    | 9     | jugador 1 | 3     | 32    | jugador 1 | greedy         |
| 10   | 13    | jugador 1 | 8     | 66    | jugador 1 | greedy         |
| 11   | 25    | jugador 1 | 8     | 50    | jugador 1 | greedy         |
| 12   | 25    | jugador 1 | 7     | 61    | jugador 1 | greedy         |
| 13   | 17    | jugador 1 | 0     | 50    | jugador 1 | greedy         |
| 14   | 9     | jugador 1 | 6     | 60    | jugador 1 | greedy         |

|    |    |           |    |    |           |        |
|----|----|-----------|----|----|-----------|--------|
| 15 | 9  | jugador 1 | 4  | 51 | jugador 1 | greedy |
| 16 | 5  | jugador 1 | 4  | 74 | jugador 1 | greedy |
| 17 | 21 | jugador 1 | 9  | 73 | jugador 1 | greedy |
| 18 | 1  | jugador 1 | 7  | 81 | jugador 1 | greedy |
| 19 | 15 | jugador 1 | 8  | 20 | jugador 1 | greedy |
| 20 | 19 | jugador 1 | 10 | 46 | jugador 1 | greedy |
| 21 | 21 | jugador 1 | 8  | 45 | jugador 1 | greedy |
| 22 | 29 | jugador 1 | 6  | 52 | jugador 1 | greedy |
| 23 | 23 | jugador 1 | 7  | 55 | jugador 1 | greedy |
| 24 | 21 | jugador 1 | 5  | 48 | jugador 1 | greedy |
| 25 | 13 | jugador 1 | 10 | 56 | jugador 1 | greedy |

## 5.2 Detalle pro vs greedy

| Seed | Iter. | Cortó     | DW J1 | DW J2 | Ganador   | Estr. ganadora |
|------|-------|-----------|-------|-------|-----------|----------------|
| 1    | 18    | jugador 2 | 47    | 7     | jugador 2 | greedy         |
| 2    | 16    | jugador 2 | 17    | 9     | jugador 2 | greedy         |
| 3    | 21    | jugador 1 | 7     | 18    | jugador 1 | pro            |
| 4    | 18    | jugador 2 | 13    | 9     | jugador 2 | greedy         |
| 5    | 15    | jugador 1 | 7     | 18    | jugador 1 | pro            |
| 6    | 21    | jugador 1 | 10    | 17    | jugador 1 | pro            |
| 7    | 13    | jugador 1 | 8     | 38    | jugador 1 | pro            |
| 8    | 10    | jugador 2 | 11    | 8     | jugador 2 | greedy         |
| 9    | 10    | jugador 2 | 26    | 9     | jugador 2 | greedy         |
| 10   | 2     | jugador 2 | 58    | 8     | jugador 2 | greedy         |
| 11   | 23    | jugador 1 | 10    | 15    | jugador 1 | pro            |
| 12   | 15    | jugador 1 | 8     | 24    | jugador 1 | pro            |
| 13   | 21    | jugador 1 | 6     | 17    | jugador 1 | pro            |
| 14   | 7     | jugador 1 | 7     | 31    | jugador 1 | pro            |
| 15   | 15    | jugador 1 | 10    | 15    | jugador 1 | pro            |
| 16   | 7     | jugador 1 | 10    | 11    | jugador 1 | pro            |
| 17   | 11    | jugador 1 | 1     | 28    | jugador 1 | pro            |
| 18   | 1     | jugador 1 | 7     | 81    | jugador 1 | pro            |
| 19   | 12    | jugador 2 | 28    | 8     | jugador 2 | greedy         |
| 20   | 13    | jugador 1 | 9     | 41    | jugador 1 | pro            |
| 21   | 14    | jugador 2 | 21    | 1     | jugador 2 | greedy         |
| 22   | 7     | jugador 1 | 1     | 20    | jugador 1 | pro            |
| 23   | 15    | jugador 1 | 5     | 12    | jugador 1 | pro            |
| 24   | 12    | jugador 2 | 19    | 6     | jugador 2 | greedy         |
| 25   | 7     | jugador 1 | 5     | 33    | jugador 1 | pro            |

## 6. Análisis de resultados

En greedy vs random, el resultado fue contundente: greedy ganó todas las partidas. Esto es esperable porque random no evalúa el efecto de sus decisiones sobre el deadwood, mientras que greedy utiliza `best_melds/4` para mantener o reducir el valor de las cartas sobrantes. Además, random puede descartar cartas útiles o continuar sin una razón estratégica.

En pro vs greedy, pro ganó 16 partidas y greedy 9. El resultado es relevante porque greedy no descarta simplemente la carta más alta entre los sobrantes, sino que evalúa cada descarte posible según el deadwood final inmediato. Por lo tanto, la comparación se realiza contra una estrategia greedy más fuerte y más consistente con la idea de optimización local.

La mejora principal de pro proviene de evaluar las manos con una función de puntaje que incorpora potencial futuro. Greedy minimiza el deadwood de la mano final de 10 cartas, mientras que pro también considera si las cartas sobrantes tienen posibilidades de formar sets o runs en turnos siguientes, descontando cartas que ya fueron vistas. Esta diferencia explica que pro gane más partidas en el conjunto de semillas utilizado.

La estrategia pro mantiene una política de cierre conservadora: corta siempre que el deadwood permite hacerlo legalmente. Esto evita prolongar partidas en situaciones donde ya existe una mano válida para cerrar, mientras que las decisiones de robo y descarte continúan utilizando una evaluación más completa que la estrategia greedy.